
SOFTWARE DESIGN SPECIFICATION

for

Encost Smart Graph Project

Version 1.0

Prepared by: Alexander Stokes
SoftFlux Engineer

SoftFlux

April 7, 2023

Contents

1	Introduction/Purpose	3
1.1	Purpose	3
1.2	Document Conventions	3
1.3	Intended Audience and Reading Suggestions	3
1.4	Project Scope	4
2	Specialized Requirements Specification	4
2.1	Dataset import format	4
2.2	User type switching	4
2.3	Household localised graphs	5
2.4	Connection limitations	5
2.5	Graph Data structure	5
2.6	Deployment to Windows devices	5
2.7	Encryption vs Hashing	5
2.8	Securely reading password	5
2.9	Connectivity stats	5
3	Assumptions	6
4	Software Architecture	6
4.1	Component Architecture	6
4.2	Deployment Diagram	7
5	Detailed Component Design	7
5.0.1	Dataset structure	8
5.1	Console Application	8
5.1.1	Default Encost Dataset	8
5.1.2	Use case diagram	8
5.1.3	Process Diagram	9
5.2	Class Diagram	14
5.2.1	Program object	14
5.2.2	UserType object	14
5.2.3	Dataset Class	15
5.2.4	Device Class	17
5.2.5	CategoryInfo Class	18
5.2.6	LocalityInfo Class	18
5.3	Graph Visualisation Window	19
5.3.1	Category Indicators	19
6	Conclusion	19

Revision History

Name	Date	Reason for Changes	Version

1 Introduction/Purpose

1.1 Purpose

The purpose of this document is to specify the software design of a software system called Encost Smart Graph Project (ESGP). This document will describe the design, architecture, and components that will form the software. This design document will be designed to fit the specified requirements.

1.2 Document Conventions

This document uses the following conventions

- ESGP: Encost Smart Graph Project
- GUI: Graphics User Interface
- CSV: Comma Separated Values

1.3 Intended Audience and Reading Suggestions

This document is intended for any developer, tester, and project manager. Here are the potential uses for each of the reader types:

- Developer: The developer who wants to read, change, modify or improve the existing program, should first consult this document and update the design in an appropriate manner so as to not destroy the actual meaning of them and pass the information correctly to the next phases of the development process.

- Tester: The tester needs this document to implement a testing specification. This document contains the structure and some aspects of the analysis of the design and architecture and is organized based on the IEEE Standard for Software Design Specification (IEEE 1016-1998).

- Project Manager: The project manager who wants to manage the parts of the project, structure their development team, or understand the design structure.

1.4 Project Scope

Encost is a new and emerging Smart Home development company. They manufacture a series of Smart Home and IoT solutions, including Wifi Routers, Smart Hubs and Controllers, Smart Light Bulbs, Smart Appliances, and Smart Whiteware. Encost is interested in investigating how their smart devices are being used and connected within households across New Zealand. They have worked, in partnership with energy companies and their users, to gather information about the smart devices that have been in use in 100 New Zealand homes between April 2020 and April 2022 (called the Encost Smart Homes Dataset). The Encost Smart Graph Project (ESGP) is a software system that enables the visualisation of Encost's devices using a graph data structure. When provided with the Encost Smart Homes Dataset, ESGP enables users to view all of the devices in the dataset, along with their connection to one another. It also provides verified users with summary statistics on device distribution, location, and connectivity.

2 Specialized Requirements Specification

2.1 Dataset import format

The original document did not specify the file input required for the dataset. Upon clarification with the client it was specified the Encost Dataset would be provided in a .txt file in a CSV data format, so the system must be designed to support this.

2.2 User type switching

During the selection of user type, the original specification did not account for the case where the user may accidentally choose to log in, or have forgotten their details. The design will be extended to allow users to return to the 'welcome' screen if they accidentally selected to log in.

2.3 Household localised graphs

It was noted through analysis of sample data that devices are localised to households. The client has confirmed that no devices will be linked across households.

2.4 Connection limitations

It was noted through analysis of the sample data that devices are only ever connected to a router. Each router will not be connected to another router based on the current specification, and there will only be a single router per household. An important note that was made by the client, is that there may be multiple range extenders which devices can connect to instead of the router directly.

2.5 Graph Data structure

An ambiguity was observed with the usage of the phrase 'graph data structure', upon discussion with the client it was clarified to mean any data structure that can store the relevant information to generate the graph.

2.6 Deployment to Windows devices

The requirements did not specify how the product is to be deployed on a device's

2.7 Encryption vs Hashing

The passwords in the system are specified in the specifications to be 'encrypted', however, upon recommendation to the client it has been decided to employ hashing instead. The client has allowed this change. Hashing is a more secure method of securing passwords, as it does not allow the password to be regained after being hashed which secures the password in the case the hash is exposed.

2.8 Securely reading password

In the interest of security, it is suggested to use a secure method to read in passwords. The system should use the Java Console's secure 'readPassword' function when reading the password, to ensure it is read securely, this was chosen because it is a native method used by the Java Console object, accessible via `System.console()`.

2.9 Connectivity stats

"Number of Smart devices that an Encost Hub/Controller is sending commands to" is interpreted to mean the average number of devices connected to each Smart Hub/Con-

troller through the router, regardless of if they can receive commands or not.

3 Assumptions

This specification assumes the following:

- Encost Employees will use the 'Encost User' accounts.
- The program will exit when the user clicks the console window's exit key.
- Future updates of the Encost dataset are out-of-scope for this Software Desing Specification.
- 'Prompts' can be any number of lines.
- The user has a basic understanding of computer use (e.g. Press 'enter' to submit a selection, know what a username and password are, etc).
- Device IDs are unique.
- "All households must be represented" means the HouseholdIDs must be marked in some way.
- The router a device connects to also sees that device as a 'connection' with itself, i.e. All connections are two-way (independent of send/receive capabilities).
- Encost Hubs/Controllers always have send capabilities.

4 Software Architecture

4.1 Component Architecture

The system is fairly simple on a larger architecture and component level as it is a locally run Java Console application with no internet requirements. The program is a console application to which the user will log in to access extra features, view a graph visualisation of the Encost dataset, etc. The other component is a GUI graph data, generated by the Java GraphStream library and is opened by the console application, but does not communicate back to the console application after it has been opened.

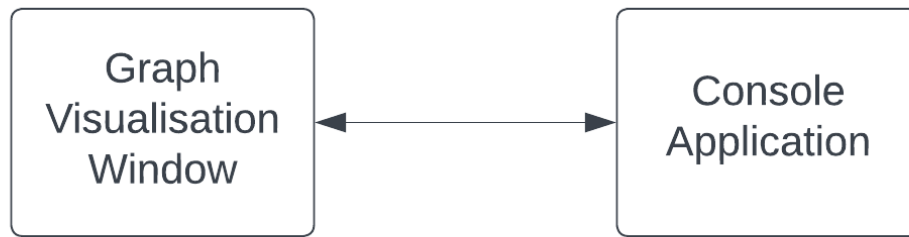


Figure 4.1: Component Architecture Diagram

4.2 Deployment Diagram

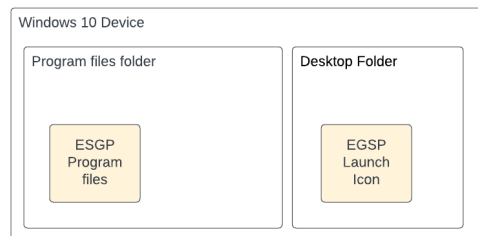


Figure 4.2: Deployment diagram

The product is a Java console application, which is compiled into a JAR file. When delivering the product, the JAR file representing the system and the EncostDataset.txt will be converted into a Windows installable application, including providing a desktop shortcut and application icon.

Launch4j is an open-source, community tool which can generate files for users to install the application on their Windows 10 system, which is the specified target operating system. The Launch4j program was chosen because it is free, well-maintained, allows adding a program icon to the installation, and provides install and uninstall capabilities. Installing the program as a Windows 10 program will allow easy use and deployment of the product.

5 Detailed Component Design

5.0.1 Dataset structure

Device ID	Date Connected	Device name	Device type	Household ID	Router Connection	Sends	Receives
EWR-1234	01/04/22	Encost Router 360	Router	WKO-1234	-	Yes	Yes
ELB-4567	01/04/22	Encost Smart Bulb B22 (multi colour)	Light bulb	WKO-1234	EWR-1234	No	Yes
EK-9876	07/05/22	Encost Smart Jug	Kettle	WKO-1234	EWR-1234	No	Yes
EHC-2468	01/04/22	Encost Smart Hub 2.0	Hub/Controller	WKO-1234	EWR-1234	Yes	Yes

Table 5.1: SRS Excerpt - Example valid dataset data and columns

The dataset will be in a CSV structure based on the above table taken from the Software Requirements Specification.

5.1 Console Application

5.1.1 Default Encost Dataset

The system will have an Encost dataset file supplied alongside the system, a .txt file in CSV format. It will be stored with the program files of the program, alongside the executable file, for easy access by the system. The dataset is read into memory just before it is needed (see 5.1.3), as the user may only want to load a custom dataset and should not have to wait for unneeded data to be processed. The file will be accessed via the 'defaultEncostFilepath' attribute in the systems main Program option which houses the main function (see 5.8).

5.1.2 Use case diagram

Figure 5.1 demonstrates how a user might use the system in a broader sense, the following section will seek to break down the individual design of each of these steps. The system has four main areas to it, user type selection, logging in, feature selection, and features

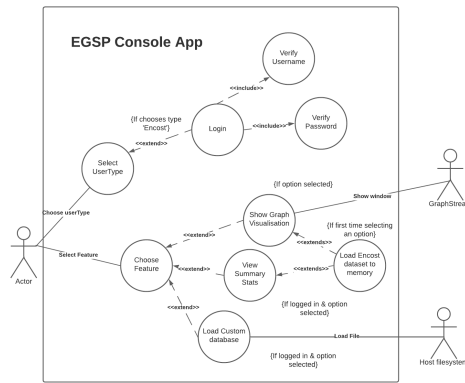


Figure 5.1: Use Case Diagram

(Graph Visualisation, Load Custom Database, and Summary Statistics). Of the two user types of the system, Community Users do not need to log in, which is why 'Log In' is an extension, which follows the software specifications. The Graph Visualisation is the feature that interacts with and opens the GUI component. The custom dataset must be fetched from the host file system of the device running the program to use this feature. Each feature is only activated when selected, which is why they are marked as extensions.

5.1.3 Process Diagram

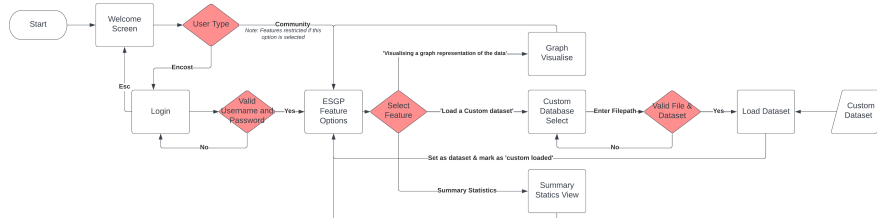


Figure 5.2: Process Diagram of system

Figure 5.2 demonstrates an overview of the general logical flow of the system, which will be further elaborated on. It defines how the user will move through the system. Features are restricted by which path the user chooses to follow, and as a result of adding to the specifications a 'go back' option will be given at any major points, such that users can undo an accidental selection. The welcome screen is the first stage of the application, and there is no endpoint until the user closes the application, which is an out-of-scope action.

Welcome screen



Figure 5.3: Sample welcome screen design

The welcome screen (Figure 5.3) is the first screen that users are greeted with on booting the application. It will first welcome the user to the application, informing them of the application's use. The welcome screen will then prompt the user to choose if they are either an Encost User or a Community User - Figure 5.3 uses "C/e" to decide, C for Community User, and E for Encost User.

- Option "C" (default choice if 'Enter' is immediately pressed) will cause the console to present the ESGP Feature Options prompt (see 5.1.3).
- Option "E" will cause the console to present the ESGP Account Login prompts (see 5.1.3).

On selection system will create a 'UserType' object (see 5.2.2), visible in the class diagram (Figure 5.8), containing the user's selection. The UserType object will handle any logging in, or modifications to the user's type and any allowed features specified by each user type.

If an invalid option is selected, the prompt should be reprinted with the text "*Invalid selection, please try again*" printed first.

ESGP Account Login

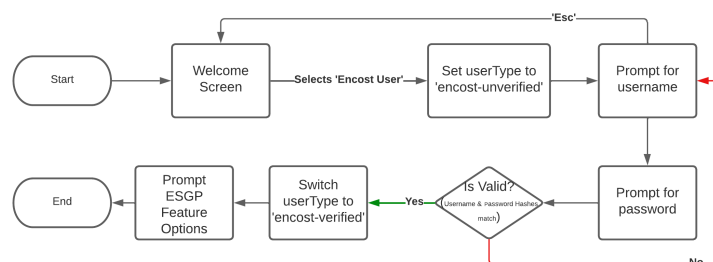


Figure 5.4: Login Process Diagram

Figure 5.4 demonstrates the logical flow of the ESGP Account Login, this assumes that the user selects 'Encost User' during the welcome screen. When prompting for the username, the option to "*Press 'Esc' to go back*" must accompany it in case the user has forgotten their account, or has accidentally selected the wrong user type, and returns the user to the welcome screen.

Checks done by the system for usernames/passwords will be done using the functions provided by the UserType object class (see 5.2.2 for justification). If the user submits an incorrect combination of username/password the user should be notified with the text "*Username/Password was incorrect. Please try again!*", and restart the prompting process. The aforementioned message informs the user clearly of the next steps to take, and does not specify which piece of information is incorrect to prevent an attacker from brute-forcing the account name, and then the password.

This logical process is specified by the specifications for the software and is designed to meet it. Username and password prompts shall be "*Username:*" and "*Password:*" respectively, which is clear and straightforward for the user. 2.8 specifies passwords should be read in via the 'readPassword' function which is accessible via the object from the 'System.console()' function.

ESGP Feature Options

```
---ESGP Feature Options---
(1) Visualise a graph representation of the data, (2) Load a custom dataset, (3) View summary statistics
(1/2/3):
```

Figure 5.5: Example feature options prompt

Figure 5.5 demonstrates what the user might see if they have access to all features.

This stage of the application will prompt the user to decide which of the allowed options they would like to use. The available options are defined by the 'getAllowedFeatures' function of the userType object, the prompt will only show available features to the user based on their type (see 5.2.2 for available features for the user types).

If the user does not select an option and presses "Enter", or enters an incorrect selection, the system must re-prompt "*Invalid option - Choose again (1/2/3):*". The system should not choose a default option, as this is a significant branching point of the program. "*(1/2/3):*" will be on a new line for readability and clear communication to the user as specified in the software requirements.

If the database object's file path is different to the default file path, the "(2)" option mentioned above must instead print "*(2) Custom database loaded - Load another*". The aforementioned prompt still allows the user to reselect a different dataset but also communicates the program's status as per the specifications.

"*Visualise a graph representation*" was put first as it is the main functionality of the system, and is the only feature available to all users. After each function has completed its tasks, the ESGP Feature Options will be prompted again so the user does not have to keep restarting the program to access more than one feature.

Graph Visualisation

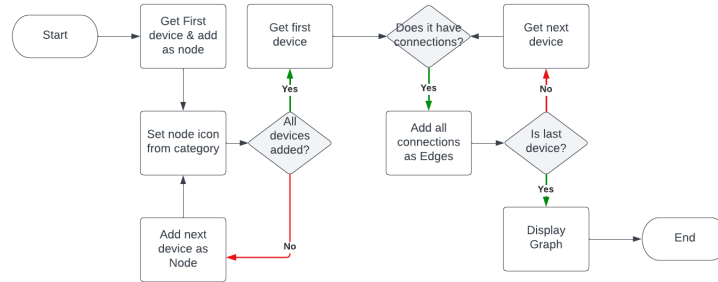


Figure 5.6: Process diagram of graph generation

This feature will use the GraphStream Java library to visualise the graph represented by the dataset class. When the user selects this feature, the 'visualise()' function will be called on the dataset object to handle the calling, graph building, and calling to the GraphStream library to show a graph window.

When generating the graph window, the devices must all be added as nodes **before** any connections are made, otherwise some nodes may not exist. Each device will set its node's icon using the icon path associated with the device's categoryInfo object (see 5.2.5). The graph will visualise the connectivity of each device through an up arrow, and a down arrow separated by a '/'. The up arrow represents a device has send capabilities and the down arrow means it can receive commands. If a device has both, the text will be separated by a '/', where the up arrow will come first. Up and down arrows were chosen as they are universally understood to represent information going OUT or IN, based on the context of the internet.

There is only one router per household, so to represent the household on the graph a label will be created above the router's node to mark the associated household ID. To prevent duplicate edges, before creating an edge the system must check if an edge ID already exists. An edge is identified by the two device IDs concatenated in alphabetical order. Device IDs are unique, so can be used as unique node and edge IDs for the GraphStream library.

If the generation takes longer than 1 second, the user must be notified in the command prompt "The Graph is being generated, this will take up to 5 seconds..."

Summary Statistics

This feature prints out all the summary information associated with the dataset loaded into memory. On the dataset object the program will call the three dataset class functions 'printLocalitySummary()', 'printDeviceDistrbutionSummary()', and 'printConnectivity-Summary()' which will handle all the information collecting and displaying for the user (see 5.2.3). After the information is displayed, the user will be prompted again with the

ESGP Feature Options.

Custom Database Select

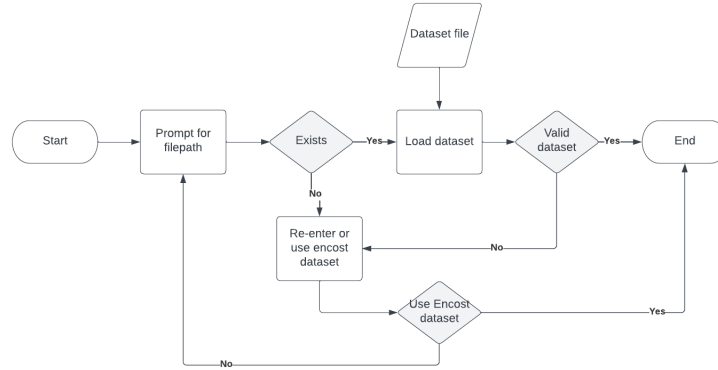


Figure 5.7: Process diagram of custom database select

The user will be prompted to enter a file path for the Microsoft Windows filesystem, for example, 'C:/User/Encost/Documents/dataset.txt'. If the file exists, the system will attempt to open it and check if the file is in CSV format and has the correct columns (see 5.1). Once the file is read, its dataset class will be created to parse the CSV information into its own structure. If the user opts to use the Encost dataset (see 5.1.1), it will only be reprocessed if the currently loaded dataset does not match with the 'defaultEncostDataset' attribute in the main Program class (see 5.1.1). Once a database is loaded it will return to the ESGP Feature Options page.

Load Dataset

To prevent the user from having to wait for the file to be processed for every single feature that requires a dataset, a dataset will be read into the system and stored as a 'dataset' object (see 5.2.3). Once the object is created the associated file will not need to be reprocessed. Because the processing will likely take more than a few seconds, the user will be informed with a console message "*Dataset is being loaded, this will take a few seconds...*". For an example of the information provided by a valid dataset, see 5.1.

If during the processing of the dataset there are some invalid entries, for example, an invalid category type of a device, the user will be notified and that individual device will be skipped. If the issue is critical to the generation of the dataset, for example, important columns are missing, the dataset will fail to be created and the user will be notified "*Database structure is invalid, would you like (1) Enter another, (2) Use the Encost dataset.*". The console messages information are to ensure the user understands what is happening, and any error messages are informative.

5.2 Class Diagram

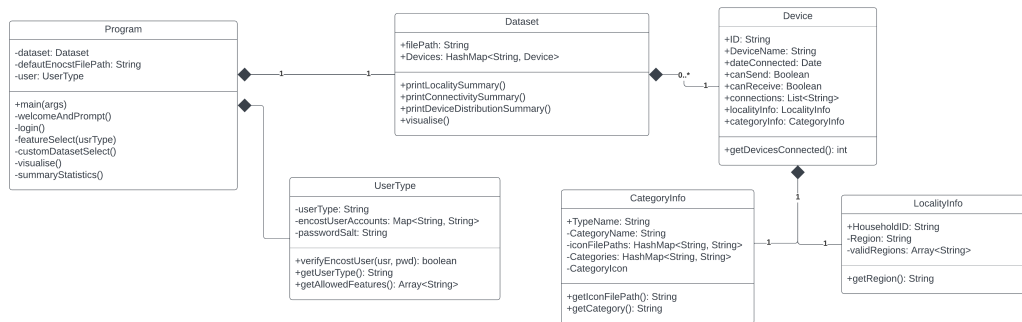


Figure 5.8: Class Diagram

This class diagram demonstrates how the console application interacts with the data structure. The program will hold a dataset of the current graph, which it can call functions to extract information on. The program itself will handle most of the application's functionalities. It was done this way so custom datasets could easily be loaded, such that graph visualisations, and summary generation is abstracted away from the main program.

Devices will handle categorising themselves, and the dataset will handle all functions related to itself. The main program function will hold functions that call all the major steps, and the user authentication will be managed by a separate class. Variables that are private in any of these classes were made so that they are either only needed by the class itself, or should publicly be read-only, to focus on security and to only provide the functionality needed.

5.2.1 Program object

This object is the default class created when the program starts, it houses the main function which is the program's entry point. This class holds all of the functions which initiate all the major steps in the program (see 5.2). All the functions are noted as private functions in the class diagram, as only the main function should have access to them. This object will hold the default file path for the Encost Dataset, which will likely be `"/"` due to the dataset being housed alongside the program's files (see 5.1.1). This path attribute fulfils the software specifications to have the Encost database in the system, and have its default file path known.

5.2.2 UserType object

This object is separate from the graph data structure, and it is held by the program as an object. It holds the user's type and will handle the verifying of a username/password pair against a set of 10 pre-defined user-password pairs provided by Encost. All passwords

must be hashed using a secure hashing algorithm, and combined with a password salt to meet the software requirements specifications for password handling. Password salts prevent malicious actors from using precomputed hashes. Password checking and account verification will be handled by this object, any users of this class will only know if a login/account verification has succeeded. Keeping the login accounts, and all login-related functions, encapsulated in an object allows the system to keep the 10 user-password pairs private and maintain the requirements of the specification for security and account login functionality.

Roles	Custom datasets	Graph visualisations	Summary statistics
Community	No	Yes	No
Encost-verified	Yes	Yes	Yes
Encost-unverified	No	No	No

Table 5.2: User types and accessible features

Table 5.2 demonstrates the available features of each user type, and what user types are available. `userType` will hold each user type as a lowercase string. 'Encost-unverified' users should always be redirected to the 'ESGP Account Login' stage, so will have zero features.

5.2.3 Dataset Class

The dataset will handle visualising itself by using the `GraphStream` library, this was done such that each individual dataset can have its own graph object built. Giving the dataset the responsibility to populate the `GraphStream` graph object, and visualise itself, keeps the concerns of the database to itself. Holding a `'filePath'` attribute helps the program distinguish if the dataset is the default database.

Four summary functions exist:

'printLocalitySummary' Function

Based on the `localityInfo` object in 5.2.6 the system will calculate stats related to geographical groupings and households. Using the region information found in a device's `localityInfo` object, and `categoryInfo` object devices can easily be grouped to generate the summary output demonstrated in Figure 5.9. Indentations are used to clearly communicate a further breakdown of a less-indented piece of information above it. The output must match the structure of Figure 5.9, as it follows the specifications functional requirements. If this takes more than one second to load, it should print to console "*Calculating locality summary, this will take up to 10 seconds...*" to fulfil the non-functional requirements.

```

--- Device Location Summary ---
Region Households breakdown:
  Waikato Households: 3
    Household 1234: 3 Device(s)
      Encost Wifi Routers: 1
      Encost Smart Lighting: 2
    Household 2314: 1 Device(s)
      Encost Wifi Routers: 1
    Household 1221: 4 Device(s)
      Encost Wifi Routers: 1
      Encost Smart Lighting: 1
      Encost Smart Appliances: 2

  Nelson Households: 2
    Household 1257: 3 Device(s)
      Encost Wifi Routers: 1
      Encost Smart Lighting: 1
      Encost Smart WhiteWare: 1
    Household 1211: 1 Device(s)
      Encost Wifi Routers: 1
    Household 1200: 1 Device(s)
      Encost Wifi Routers: 1
  Bay of Plenty Households: 1
    Household 1000: 2 Device(s)
      Encost Wifi Routers: 1
      Encost Smart Appliances: 1

Region Category Breakdown
  Waikato:
    Encost Wifi Routers: 3
    Encost Smart Lighting: 3
    Encost Smart Appliances: 2

  Nelson:
    Encost Wifi Routers: 3
    Encost Smart Lighting: 1
    Encost Smart WhiteWare: 1

  Bay of Plenty:
    Encost Wifi Routers: 1
    Encost Smart Appliances: 1

```

Figure 5.9: Example location summary output

```

--- Device Distribution Summary ---
Encost Wifi Routers: 5
  Router: 4
  Extender: 1
Encost Hubs/controllers: 3
  Hub: 2
  Controllers: 1
Encost Smart Lighting: 0
Encost Smart Appliances: 0
Encost Smart WhiteWare: 0

```

Figure 5.10: Device distribution example output

'printDeviceDistributionSummary' Function

This feature prints out the breakdown of devices in each category, and the distribution of types in each category. Figure 5.10 shows an example output based on the categories found in 5.3. Fetching and grouping the categoryInfo objects in each device allows each grouping with all the relevant information. Indentations are used to clearly communicate a further breakdown of a less-indented piece of information above it. First grouping devices by the category ID, and then further grouping the devices by device types Figure 5.10 stats can be generated. The output must match the structure of Figure 5.10, as it follows the specifications functional requirements. If this takes more than one second to load, it should print to console ”*Calculating device distribution summary, this will take*

up to 10 seconds...” to fulfil the non-functional requirements.

'printConnectivitySummary' Function

```
--- Device Connectivity Summary ---
Average # devices Encost Wifi Routers are connected to: 3
  Minimum #: 0
  Maximum #: 5
Average # of Encost Hubs/Controllers devices are receiving from: 1
  Minimum #: 0
  Maximum #: 2
Average # of devices Encost Hubs/Controllers sending to: 2
  Minimum #: 0
  Maximum #: 4
```

Figure 5.11: Device connectivity example output

Based on the connections in the graph, connection information can be calculated. The system can easily calculate the first section by using the connections list in each Encost Wifi Router category device. The system can calculate the second and third sections by grouping devices by household and calculating the number of Encost Hubs/Controllers that can send against the number of devices with a category with the word 'Smart' in it with or without receiving capabilities. The second section is interpreted to provide the average of Encost Hubs/Controllers that every 'Smart' category device (with receiving capabilities) is in the same household. The third section is interpreted to provide the average of Encost Hubs/Controllers connected via a household to the number of 'Smart' category devices. It is important for section three that each 'Smart' category device is counted multiple times for each 'Encost Hubs/Controllers' connected to it in one household, and vice versa for section two. Indentations are used to clearly communicate a further breakdown of a less-indented piece of information above it. The output must match the structure of Figure 5.11. The output must match the structure of Figure 5.11, as it follows the specifications' functional requirements. If this takes more than one second to load, it should print to console *"Calculating connectivity summary, this will take up to 10 seconds..."* to fulfil the non-functional requirements.

5.2.4 Device Class

The device class will handle all its own connection information, the ability to send and receive, and the date connected. It will have a deviceInfo attribute which will hold the category, type and model information specific to the device. The canSend and canReceive booleans will be used to determine if a device can send or receive commands, which is important to the summary generation feature. Any devices that are connected to one device will use the device's ID to look up the device with the dataset's Devices hashmap and add themselves to the device's connections to ensure the edge is a two-way connection. Having a two-way connection correctly enables connection statistics to work across devices and output the required statistics in a performant way, otherwise devices may miss connections and require extra processing to confirm.

Device Category	Device types	Device Names
Encost Wifi Routers	Router, Extender	Encost Router 360, Encost Router Plus, Encost Wifi Range Extender 1.0, Encost Wifi Range Extender 2.0
Encost Hubs/- Controllers	Hub/Controller	Encost Smart Hub, Encost Smart Hub 2.0, Encost Smart Hub Mini
Encost Smart Lighting	Light Bulb, Strip Lighting, Other Lighting	Encost Smart Bulb B22 (white), Encost Smart Bulb B22 (multi colour), Encost Smart Bulb E26 (white), Encost Smart Bulb E26 (multi colour), Encost Strip Lighting (white), Encost Strip Lighting (multi colour), Encost Novelty Light (giraffe), Encost Novelty Light (lion), Encost Novelty Light (bear)
Encost Smart Appliances	Kettle, Toaster, Coffee Maker	Encost Smart Jug, Encost Smart Whistling Kettle, Encost Smart Toaster (2 slice), Encost Smart Toaster (4 slice), Encost Smart Coffee Maker, Encost Smart Coffee Maker Mini, Encost Smart Coffee Maker Pro
Encost Smart Whiteware	Washing Machine/Dryer, Refrigerator/Freezer, Dishwasher	Encost Smart Washer, Encost Smart Washer Pro, Encost Smart Dryer, Encost Smart Dryer Pro, Encost Smart Refrigerator, Encost Smart Freezer, Encost Smart Refrigerator/Freezer Combo, Encost Dishwasher, Encost Dishwasher Pro

Table 5.3: SRS Excerpt - Device category, types, and names breakdown

5.2.5 CategoryInfo Class

The input dataset columns do not include a category column, so the device category must be determined from the device type using the associations defined in Table 5.3. Using a hashmap with the device type as a key, we can find the associated category of each device. Each category has a different icon associated with it to improve the graph visualisation of what category each device is, this can be retrieved with the 'getIconFilePath()'. getIconFilePath uses a Hashmap with the category name as a key to find the associated path, a hashmap is used for speedy lookups.

5.2.6 LocalityInfo Class

This class will determine the region of a device, based on the Household ID by extracting the first three characters in the ID and comparing it against the valid NZ Regions stored in an array. This functionality is separated from the device to only implement relevant concerns in the device class. Region information is only accessible via a 'getRegion()' function to ensure the program correctly manages the region and only exposes relevant functionality to other classes. As per the specifications, the region names and codes used are taken from the ISO 3166-2 standard published by the ISO 3166 Maintenance

Agency (ISO 3166/MA)¹;

5.3 Graph Visualisation Window

As per the specifications, this component will utilise the Java GraphStream library to form the graph view visible to users. This will be a component called by the Dataset class when the 'visualise()' function has been called on a specific dataset. The dataset will manage the generation and population of the GraphStream library, which will then be 'displayed' with the underlying swing.UI library allows a GUI to appear as per the specifications. The GUI will show for the user, however, it will be a read-only view where the user can look around and zoom in to see device connectivity. The user can choose to exit, and the window will close.

5.3.1 Category Indicators



Figure 5.12: Example icon for the Encost Smart Appliance category

As per the specifications, a device must communicate what type of category it is. Icons will be determined by an 'iconFilePath' string provided by each device, which will point to a different icon image to use. The images will be displayed over each device node and will communicate to the user what category the device is under. There are various labels which will appear over the devices in the graph, such as an up and down arrow to communicate connectivity and the household ID (see . Each of these visualisations is specified in the software requirements.

6 Conclusion

Using an executable packaged Java application, this architecture will provide a simple-to-use locally run system which allows users to use graph visualisation and gain summary statistics of the graph. This design also allows for custom datasets to be visualised and analysed. This software design specification uses a use case diagram, class diagram, and a deployment diagram, paired with process diagrams to illustrate the design of the system and how it meets the software requirements specification. It has laid out how the logic will work to meet performance requirements, and it has also laid out how

¹https://en.wikipedia.org/wiki/ISO_3166-2:NZ

users can be easily prompted for information and informed of any errors. This Software Design Specification has satisfied the purpose, product function, software architecture, and component design for the Encost Smart Graph Project (ESGP) for the company Encost.